43

日志结构文件系统

20 世纪 90 年代初，伯克利的一个小组在 John Ousterhout 教授和研究生 Mendel Rosenblum 的带领下，开发了一种新的文件系统（file system），叫作日志结构文件系统（log-structured file system）[RO91]。促使他们这样做的，是以下几点观察：

- **系统内存在不断增长：**内存越大，能缓存（cache）在内存中的数据就越多。缓存的数据越多，读请求就越多地由缓存来满足，磁盘流量中写入所占的比例也就越来越大。因此，文件系统的性能在很大程度上取决于它的写入性能。
- **随机 I/O 性能和顺序 I/O 性能之间差距很大：**这些年来，硬盘的传输带宽（bandwidth）增长了很多 [P98]；盘面（surface）上塞进的位越多，访问这些位时的带宽就越高。然而，寻道（seek）和旋转延迟（rotational delay）的开销却下降得很慢；要让便宜的小电机把盘片（platter）转得更快、或者把磁盘臂（disk arm）移动得更快，是很难的。因此，如果你能顺序地使用磁盘，性能就会比那些引起寻道和旋转的用法好得多。
- **现有的文件系统在许多常见工作负载下表现不佳：**例如，FFS [MJLF84] 创建一个大小为一个块（block）的新文件时，要执行大量写入：一次写新的 inode，一次更新 inode 位图（inode bitmap），一次写文件所在目录（directory）的数据块（data block），一次更新目录的 inode，一次写属于新文件的新数据块，还有一次写数据位图（data bitmap），把这个数据块标记为已分配。因此，虽然 FFS 把所有这些块都放在同一个块组（block group）里，它还是会引起许多短距离的寻道以及随之而来的旋转延迟（rotational delay），性能因此远远达不到顺序带宽的峰值。
- **文件系统不了解 RAID：**例如，RAID-4 和 RAID-5 都有**小写入问题**（small-write problem），即对单个块的一次逻辑写入会引起 4 次物理 I/O。现有的文件系统并不会设法避开 RAID 这种最坏情况下的写入行为。

提示：细节很重要

所有有意思的系统，都是由几个一般性的思想加上大量细节组成的。有时，你在学习这些系统时会对自己说：“哦，我懂大体思路了，剩下的不过是细节。”然后你就凭这个只学会了一半，没弄清楚系统真正是怎么工作的。别这样！很多时候，细节是至关重要的。我们在 LFS 中会看到，大体思路很容易理解，但要真正构建一个能用的系统，你必须把所有棘手的情况都想清楚。

因此，一个理想的文件系统应该专注于写入性能，并设法利用磁盘的顺序带宽。此外，它在常见的工作负载（workload）下也应该表现良好，这些工作负载不仅要写出数据，还会频繁地更新磁盘上的元数据结构（metadata structure）。最后，它在 RAID 和单个磁盘上都应该工作得很好。

Rosenblum 和 Ousterhout 提出的这种新型文件系统叫作 **LFS**，是**日志结构文件系统**（Log-structured File System）的缩写。向磁盘写入时，LFS 先把所有更新都缓冲在内存中的一个**段**（segment）里，连元数据（metadata）也不例外；段满了以后，就通过一次长长的顺序传输，把它写到磁盘上一块未使用的区域。LFS 从不覆盖已有的数据，而是总是把段写到空闲的位置。由于段很大，磁盘（或 RAID）得到了高效的利用，文件系统的性能也接近其巅峰。

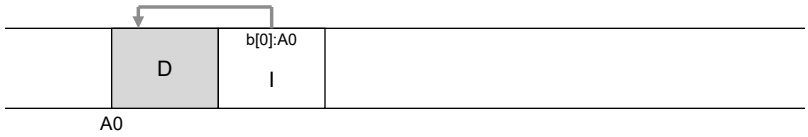
关键问题：
如何让所有写入都变成顺序写入？
文件系统怎样才能把所有写入都变成顺序写入?对于读, 这个任务是不可能完成的, 因为要读的块可能在磁盘上的任何地方。然而, 对于写, 文件系统总是有选择的余地, 而我们希望利用的正是这一选择。

43.1 顺序地写入磁盘

于是，我们面临的第一个挑战是：如何把对文件系统状态的所有更新，都变成一系列对磁盘的顺序写入？为了更好地理解这一点，我们来看一个简单的例子。假设我们要往一个文件里写一个数据块 *D*。把这个数据块写到磁盘上，可能会得到下面的磁盘布局，*D* 写在磁盘地址（disk address） *A*0 处：



然而，用户写入一个数据块时，写到磁盘上的不只是数据；还有其他一些需要更新的**元数据**（metadata）。在这个例子中，我们把文件的 **inode**（*I*）也写到磁盘上，并让它指向数据块 *D*。写到磁盘上之后，数据块和 inode 看起来大致是这样的（注意，图中 inode 看起来和数据块一样大，一般情况并非如此；在大多数系统中，数据块的大小是 4 KB，而 inode 要小得多，大约 128 字节）：



这个基本思想，即简单地把所有更新（比如数据块、inode 等）顺序地写到磁盘上，就是 LFS 的核心。理解了这一点，你就掌握了基本思路。但正如所有复杂的系统一样，魔鬼藏在细节里。

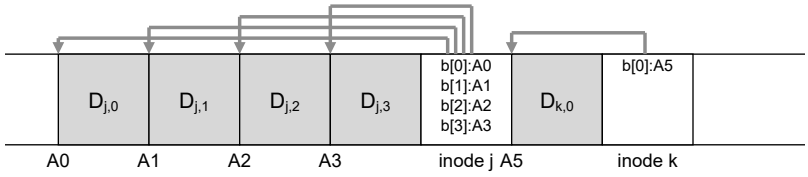
43.2 顺序而高效地写入

遗憾的是，仅仅顺序地写入磁盘，还不足以保证写入是高效的。例如，假设我们在时刻 T 向地址 A 写了一个块。然后我们等了一小会儿，在时刻 $T+\delta$ 向紧接着的下一个块地址（block address）$A+1$ 写入。不巧的是，在第一次写入和第二次写入之间，磁盘已经转过去了；当你发出第二次写入时，它要等差不多一整圈才能提交（具体来说，如果转一圈需要时间 $T_{rotation}$，磁盘就要等 $T_{rotation}-\delta$ 才能把第二次写入提交到盘面上）。由此你大概可以看出，仅仅按顺序写入磁盘，并不足以达到峰值性能；你必须向驱动器发出大量连续的写入（或者一次大的写入），才能获得良好的写入性能。

为了做到这一点，LFS 使用了一种古老的技术，叫作**写缓冲**（write buffering）[1]。在写入磁盘之前，LFS 先在内存中记录下各项更新；收到足够多的更新后，再一次性把它们全部写到磁盘上，从而确保高效地使用磁盘。

LFS 一次写入的这一大块更新，叫作**段**（segment）。虽然这个术语在计算机系统中被用得太多了，但在这里，它只是指 LFS 用来把写入归成一组的一个比较大的块。因此，向磁盘写入时，LFS 先把更新缓冲在内存中的段里，然后一次性把这个段写到磁盘上。只要段足够大，这些写入就会很高效。

下面是一个例子，LFS 把两组更新缓冲进一个较小的段里；实际的段要大得多（几 MB）。第一组更新是对文件 j 的四个块的写入；第二组是给文件 k 添加一个块。然后，LFS 一次性把这个由七个块组成的整个段提交到磁盘上。这些块最终在磁盘上的布局如下：



[1]说真的，这个思想很难找到一个合适的出处，因为它很可能在计算机历史的很早期就被许多人各自发明过。关于写缓冲（write buffering）的好处，可以参阅 Solworth 和 Orji 的研究 [SO90]；想了解它可能带来的坏处，可以参阅 Mogul 的论文 [M94]。

43.3 要缓冲多少?

这就引出了下面的问题: LFS 在写入磁盘之前, 应该缓冲多少更新? 答案当然取决于磁盘本身, 具体来说, 取决于定位开销和传输速率相比有多高; 类似的分析可以参阅 FFS 一章。

例如, 假设每次写入之前的定位, 也就是旋转和寻道开销, 大约需要 $T_{position}$ 秒。再假设磁盘的传输速率是 R_{peak} MB/s。在这样的磁盘上, LFS 在写入之前应该缓冲多少数据?

思考这个问题的方法是: 每次写入, 你都要付出一笔固定的定位开销。那么, 你要写多少数据, 才能**平摊**(amortize)这笔开销? 写得越多越好(这是显然的), 也就越接近峰值带宽。

为了得到一个具体的答案, 假设我们要写出 D MB。写出这块数据的时间(T_{write})等于定位时间 $T_{position}$ 加上传输 D 的时间($\frac{D}{R_{peak}}$), 即:

$$T_{write} = T_{position} + \frac{D}{R_{peak}} \tag{43.1}$$

因此, 有效写入速率($R_{effective}$)就是写入的数据量除以写入所花的总时间:

$$R_{effective} = \frac{D}{T_{write}} = \frac{D}{T_{position} + \frac{D}{R_{peak}}}. \tag{43.2}$$

我们关心的是让有效速率($R_{effective}$)接近峰值速率。具体来说, 我们希望有效速率是峰值速率的某个比例 F, 其中 $0 < F < 1$(典型的 F 可能是 0.9, 即峰值速率的 90%)。用数学形式表达, 就是我们希望 $R_{effective} = F \times R_{peak}$。

这样, 我们就可以解出 D:

$$R_{effective} = \frac{D}{T_{position} + \frac{D}{R_{peak}}} = F \times R_{peak} \tag{43.3}$$

$$D = F \times R_{peak} \times \left(T_{position} + \frac{D}{R_{peak}} \right) \tag{43.4}$$

$$D = \left(F \times R_{peak} \times T_{position} \right) + \left(F \times R_{peak} \times \frac{D}{R_{peak}} \right) \tag{43.5}$$

$$D = \frac{F}{1 - F} \times R_{peak} \times T_{position} \tag{43.6}$$

我们来算一个例子: 磁盘的定位时间是 10 毫秒, 峰值传输速率是 100 MB/s; 假设我们希望有效带宽达到峰值的 90%($F = 0.9$)。在这种情况下,
$D = \frac{0.9}{0.1} \times 100 \text{ MB/s} \times 0.01 \text{ 秒} = 9 \text{ MB}$。试试不同的取值, 看看需要缓冲多少数据才能接近峰值带宽。要达到峰值的 95% 需要多少? 99% 呢?

43.4 问题: 找到 inode

为了理解在 LFS 中如何找到 inode, 我们先简单回顾一下在典型的 UNIX 文件系统中如何找到 inode。在 FFS 这样的典型文件系统中, 甚至在老的 UNIX 文件系统中, 找 inode 都很容易, 因为它们组织成数组, 放在磁盘上固定的位置。

 © 2008–23, ARPACI-DUSSEAU

例如，老的 UNIX 文件系统把所有 inode 都放在磁盘的一个固定区域。因此，给定 inode 号（inode number）和起始地址，要找到某个 inode，只需把 inode 号乘以 inode 的大小，再加上磁盘上这个数组的起始地址，就能算出它确切的磁盘地址；给定 inode 号，基于数组的索引既快又直接。

在 FFS 中，根据 inode 号找 inode 只是稍微复杂一点，因为 FFS 把 inode 表（inode table）分成了若干块，在每个柱面组（cylinder group）里放一组 inode。因此，你必须知道每一块 inode 有多大，以及每一块的起始地址。之后的计算是类似的，也很简单。

在 LFS 中，事情就困难多了。为什么？嗯，我们已经把 inode 分散到了整个磁盘上！更糟的是，我们从不原地覆盖，因此 inode 的最新版本（也就是我们想要的那个）会不断移动。

43.5 用间接层来解决：inode map

为了解决这个问题，LFS 的设计者用一个叫作 **inode map**（**imap**）的数据结构（data structure），在 inode 号和 inode 之间引入了一个**间接层**（level of indirection）。imap 这个结构以 inode 号为输入，给出这个 inode 最新版本的磁盘地址。因此，你可以想象它通常会实现成一个简单的数组，每个条目 4 个字节（一个磁盘指针）。每当有 inode 写到磁盘上，imap 就会更新为它的新位置。

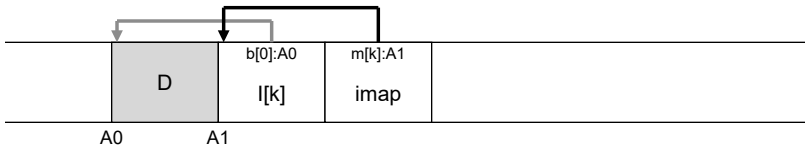
遗憾的是，imap 需要持久保存（也就是写到磁盘上）；这样做能让 LFS 在崩溃之后仍然知道各个 inode 的位置，从而按预期运行。于是有一个问题：imap 应该放在磁盘上的什么地方？

当然，它可以放在磁盘上的一个固定位置。遗憾的是，imap 更新得很频繁，这样一来，每次更新文件结构之后都得再写一次 imap，性能就会受损（每次更新之后都要多一次寻道，去 imap 所在的固定位置）。

于是，LFS 把 inode map 的各个片段直接放在其他新写入的信息旁边。因此，往文件 k 追加一个数据块时，LFS 实际上会把新数据块、它的 inode 以及 inode map 的一个片段一起写到磁盘上，如下所示：

提示：使用间接层

人们常说，计算机科学中所有问题的解决办法，就是加一个**间接层**（level of indirection）。这显然不对；它只是大多数问题的解决办法（没错，这么说还是太绝对了，但你明白我们的意思）。你当然可以把我们学过的每一种虚拟化（virtualization），比如虚拟内存（virtual memory）或文件的概念，都看作一个间接层（level of indirection）。而 LFS 中的 inode map 当然就是对 inode 号的一种虚拟化。希望你能从这些例子中看出间接层（level of indirection）的强大威力：它让我们可以自由地移动各种结构（比如虚拟内存例子中的页（page），或者 LFS 中的 inode），而不必修改对它们的每一处引用。当然，间接层（level of indirection）也有缺点：**额外的开销**。所以，下次你遇到问题时，试试用间接层（level of indirection）来解决，但一定要先想想这样做的开销。正如 Wheeler 的那句名言："计算机科学中的所有问题都可以通过再加一个间接层（level of indirection）来解决，当然，间接层太多这个问题除外。"



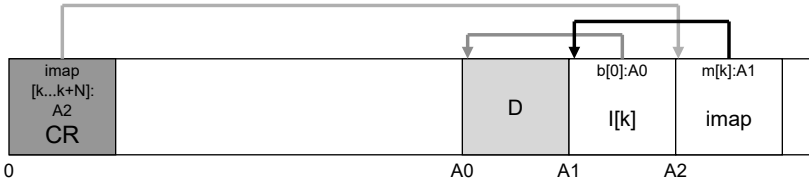
在这张图中，标着 *imap* 的那个块里存放着 imap 数组的一个片段，它告诉 LFS，inode *k* 在磁盘地址 *A*1 处；而这个 inode 又告诉 LFS，它的数据块 *D* 在地址 *A*0 处。

43.6 完成方案：检查点区域

聪明的读者（就是你，对吧?）可能已经注意到这里有个问题。既然 inode map 的各个片段现在也分散在磁盘各处，我们怎么找到 inode map 呢? 归根结底，这里没有什么魔法：文件系统必须在磁盘上有某个固定的、已知的位置，作为查找文件的起点。

LFS 在磁盘上正好有这样一个固定的位置，叫作**检查点区域**（checkpoint region，CR）。检查点区域（checkpoint region）包含指向 inode map 最新片段的指针（即它们的地址），因此先读 CR 就能找到 inode map 的各个片段。注意，检查点区域（checkpoint region）只是定期更新（比如每 30 秒左右一次），因此性能不会受到不良影响。于是，磁盘布局的整体结构包含一个检查点区域（checkpoint region），它指向 inode map 的最新片段；inode map 的每个片段包含 inode 的地址；inode 则像典型的 UNIX 文件系统中那样指向文件和目录。

下面是一个例子，其中有检查点区域（checkpoint region，注意它一直放在磁盘的最开头，地址 0 处），以及一个 imap 片段、一个 inode 和一个数据块。真实的文件系统当然会有一个大得多的 CR（实际上会有两个，我们后面会明白为什么）、许多 imap 片段，当然还有多得多的 inode、数据块等等。



43.7 从磁盘读取文件：回顾

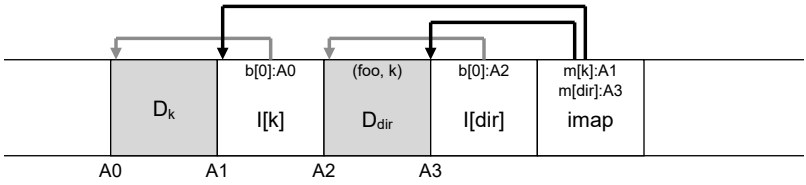
为了确保你理解了 LFS 的工作原理，我们现在来逐步看看从磁盘读取一个文件必须经过哪些步骤。假设一开始内存里什么都没有。我们必须读取的第一个磁盘数据结构是检查点区域（checkpoint region）。检查点区域（checkpoint region）包含指向整个 inode map 的指针，也就是这些片段的磁盘地址，于是 LFS 接着读入整个 inode map，并把它缓存在内存中。此后，给定一个文件的 inode 号，LFS 只需在 imap 中查找 inode 号到 inode 磁盘地址的映射，然后读入这个 inode 的最新版本。这时，要读文件中的某个块，LFS 的做法

就和典型的 UNIX 文件系统完全一样，根据需要使用直接指针（direct pointer）、间接指针（indirect pointer）或二级间接指针（double indirect pointer）。在常见情况（common case）下，LFS 从磁盘读取文件时执行的 I/O 次数应该和典型的文件系统相同；整个 imap 都缓存在内存里，因此 LFS 在读取时多做的工作，就是在 imap 中查找 inode 的地址。

43.8 目录怎么办?

到目前为止，我们只考虑了 inode 和数据块，把讨论稍微简化了一些。然而，要访问文件系统中的一个文件（比如 `/home/remzi/foo`，这是我们最喜欢用的虚构文件名之一），还必须访问一些目录。那么，LFS 是如何存储目录数据的呢?

幸运的是，目录结构和经典的 UNIX 文件系统基本相同：目录只是（名字，inode 号）映射的集合。例如，在磁盘上创建一个文件时，LFS 既要写入一个新的 inode、一些数据，也要写入引用这个文件的目录数据及其 inode。记住，LFS 会先把这些更新缓冲一段时间，再把它们顺序地写到磁盘上。因此，在某个目录中创建文件 `foo`，会在磁盘上产生下面这些新结构：



inode map 的这个片段里，同时包含目录文件 *dir* 和新创建的文件 *f* 的位置信息。因此，访问 inode 号为 *k* 的文件 `foo` 时，你会先在 inode map（通常缓存在内存中）里查找目录 *dir* 的 inode 的位置（*A*3）；然后读取目录的 inode，它给出目录数据的位置（*A*2）；读取这个数据块，就得到名字到 inode 号的映射（`foo`, *k*）。接着再查一次 inode map，找到 inode 号 *k* 的位置（*A*1），最后读取地址 *A*0 处所需的数据块。

inode map 还解决了 LFS 中另一个严重的问题，叫作**递归更新问题**（recursive update problem）[Z+12]。任何从不原地更新、而是把更新移到磁盘上新位置的文件系统（比如 LFS），都会出现这个问题。

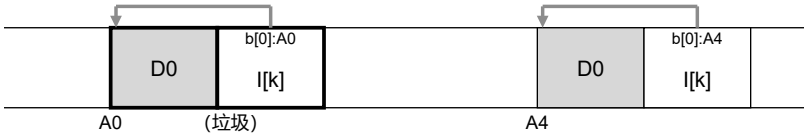
具体来说，每当更新一个 inode，它在磁盘上的位置就会改变。稍不注意，指向这个文件的目录也得跟着更新一次，而这又要求修改这个目录的父目录（parent directory），如此一路向上，直到文件系统树（file system tree）的根。

LFS 用 inode map 巧妙地避开了这个问题。尽管 inode 的位置可能会变，这种变化却永远不会反映到目录本身；相反，更新的是 imap 这个结构，而目录里保存的名字到 inode 号的映射保持不变。因此，借助间接层（level of indirection），LFS 避免了递归更新问题（recursive update problem）。

43.9 新的问题：垃圾回收

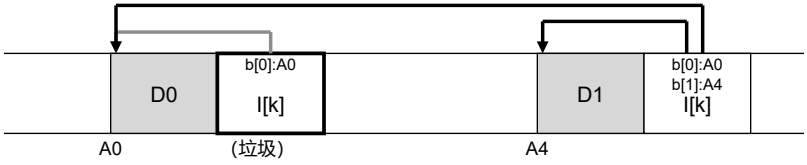
你可能已经注意到 LFS 的另一个问题：它不断地把文件的最新版本（包括 inode 和数据）写到磁盘上的新位置。这个过程虽然让写入保持高效，却意味着 LFS 会在磁盘各处留下文件结构的旧版本。我们（相当不客气地）把这些旧版本叫作**垃圾**（garbage）。

例如，假设有一个已存在的文件，inode 号为 k，它指向一个数据块 $D0$。现在我们更新这个块，产生一个新的 inode 和一个新的数据块。LFS 最终在磁盘上的布局大致是这样的（注意，为简单起见，我们省略了 imap 和其他结构；实际上还必须把一个新的 imap 片段写到磁盘上，指向新的 inode）：



从图中可以看到，inode 和数据块在磁盘上都有两个版本，一个是旧的（左边那个），一个是当前的、因而是**存活**（live）的（右边那个）。只是在逻辑上更新了一个数据块这么简单的操作，LFS 就必须持久保存若干个新结构，从而在磁盘上留下这些块的旧版本。

再举一个例子，假设我们改为往原来的文件 k 追加一个块。在这种情况下，会生成 inode 的一个新版本，但旧数据块仍然被这个 inode 指着。因此，它仍然是存活的，仍然是当前文件系统的一部分：



那么，我们该怎么处理这些较老版本的 inode、数据块等等呢？一种办法是把这些较老的版本保留下来，让用户可以恢复文件的旧版本（例如，用户不小心覆盖或删除了文件时，这样做会非常方便）；这样的文件系统叫作**版本化文件系统**（versioning file system），因为它记录着文件的不同版本。

然而，LFS 只保留文件最新的存活版本；因此，LFS 必须在后台定期找出文件数据、inode 和其他结构的这些已经死亡的旧版本，并**清理**（clean）它们；清理应该让磁盘上的块重新空闲出来，供后续写入使用。注意，清理其实是**垃圾回收**（garbage collection）的一种形式；有些编程语言会自动替程序释放不再使用的内存，用的也是这种技术。

前面我们讨论过段的重要性：正是段让 LFS 能够向磁盘大块地写入。事实证明，段对于有效的清理也至关重要。想象一下，如果 LFS 的清理器（cleaner）在清理时只是逐个释放单个数据块、inode 等，会发生什么。结果是：文件系统在磁盘上的已分配空间之间夹杂

 © 2008–23, ARPACI-DUSSEAU

着一些空闲的**空洞**（hole）。写入性能会大幅下降，因为 LFS 找不到一大片连续的区域来做顺序的高性能写入。

因此，LFS 的清理器（cleaner）是逐段工作的，从而为后续写入腾出大块的空间。基本的清理过程如下。LFS 清理器（cleaner）定期读入一些旧的（部分使用的）段，确定这些段中哪些块是存活的，然后写出一组新的段，里面只包含存活的块，从而释放旧段（old segment）供写入使用。具体来说，我们希望清理器（cleaner）读入 M 个已有的段，把它们的内容**紧凑**（compact）到 N 个新段（new segment）中（其中 $N < M$），然后把这 N 个段写到磁盘上的新位置。随后，旧的 M 个段就被释放，文件系统可以用它们来做后续的写入。

不过，现在我们还剩下两个问题。第一个是机制（mechanism）问题：LFS 怎样判断段中的哪些块是存活的、哪些是死亡的？第二个是策略（policy）问题：清理器（cleaner）应该多久运行一次，应该挑选哪些段来清理？

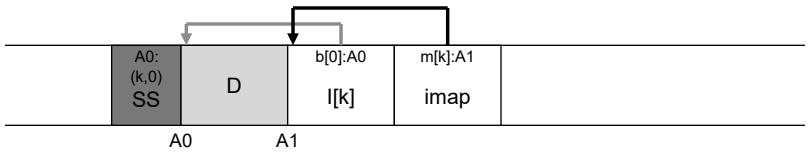
43.10 判断块是否存活

我们先来解决机制问题。给定磁盘段 S 中的一个数据块 D，LFS 必须能判断 D 是否存活。为此，LFS 给每个段加了一点额外的信息，用来描述每个块。具体来说，对于每个数据块 D，LFS 记录了它的 inode 号和偏移量（offset，即它是文件的第几个块）。这些信息记录在段开头的一个结构中，叫作**段摘要块**（segment summary block）。

有了这些信息，判断一个块是存活还是死亡就很直接了。对于位于磁盘地址 A 的块 D，在段摘要块（segment summary block）中查出它的 inode 号 N 和偏移量 T。接着，在 imap 中找到 N 所在的位置，从磁盘读出 N（它也许已经在内存里了，那就更好了）。最后，用偏移量 T 查看 inode 或某个间接块（indirect block），看看 inode 认为这个文件的第 T 个块在磁盘上的什么位置。如果它恰好指向磁盘地址 A，LFS 就可以断定块 D 是存活的。如果它指向别的地方，LFS 就可以断定 D 已不再使用（即它是死亡的），从而知道不再需要这个版本了。下面是伪代码（pseudocode）总结：

```
(N, T) = SegmentSummary[A];
inode  = Read(imap[N]);
if (inode[T] == A)
    // 块 D 是存活的
else
    // 块 D 是垃圾
```

下图描述了这一机制，其中段摘要块（segment summary block，标为 *SS*）记录着地址 *A0* 处的数据块实际上属于文件 k，偏移量为 0。查看 imap 中的 k，就能找到这个 inode，并看到它确实指向那个位置。



LFS 还用了一些捷径，让判断存活的过程更高效。例如，当文件被截断或删除时，LFS 会增加它的**版本号**（version number），并把新的版本号记录在 imap 中。LFS 同时也把版本号记录在磁盘上的段里，这样只要比较磁盘上的版本号和 imap 中的版本号，就能跳过上面描述的那种较长的检查，避免额外的读取。

43.11 策略问题：清理哪些块，何时清理？

在上述机制的基础上，LFS 还必须包含一组策略，来决定何时清理，以及哪些块值得清理。决定何时清理比较容易：可以定期清理，可以在空闲时清理，也可以在磁盘满了、不得不清理的时候清理。

决定清理哪些块则更有挑战性，许多研究论文都以此为题。在最初的 LFS 论文 [RO91] 中，作者描述了一种设法区分热段（hot segment）和冷段（cold segment）的方法。热段（hot segment）是指其中的内容经常被覆盖的段；因此，对于这样的段，最好的策略是等很长时间再去清理它，因为这期间会有越来越多的块被覆盖、写进新段（new segment），从而释放出来供使用。相比之下，冷段（cold segment）可能有少数几个死亡的块，但其余内容相对稳定。因此，作者得出的结论是，应该早点清理冷段（cold segment）、晚点清理热段（hot segment），并设计了一个正是这样做的启发式方法。然而，和大多数策略一样，这个策略并不完美；后来的方法展示了如何做得更好 [MR+97]。

43.12 崩溃恢复和日志

最后一个问题：如果系统在 LFS 写入磁盘时崩溃了，会怎样？你可能还记得上一章讲日志（journaling）时说过，更新过程中发生崩溃对文件系统来说很棘手，因此 LFS 也必须考虑这个问题。

在正常运行时，LFS 把写入缓冲在段中，等段满了或者过了一段时间，再把段写到磁盘上。LFS 把这些写入组织成一个**日志**（log），即检查点区域（checkpoint region）指向头段（head segment）和尾段（tail segment），每个段又指向下一个要写的段。LFS 还会定期更新检查点区域（checkpoint region）。显然，写段和写 CR 的过程中都可能发生崩溃。那么，LFS 如何处理写这些结构时发生的崩溃呢？

我们先说第二种情况。为了确保 CR 的更新是原子的，LFS 实际上保留了两个 CR，分别位于磁盘的两端，并交替写入它们。用指向 inode map 的最新指针和其他信息更新 CR 时，LFS 还实现了一个谨慎的协议（protocol）；具体来说，它先写出一个头部（带有时间戳），然后写 CR 的主体，最后再写一个块。如果系统在更新 CR 的过程中崩溃，LFS 能从

前后两个时间戳不一致察觉出来。LFS 总是选择使用时间戳一致的最新的 CR，从而实现了 CR 的一致更新。

现在我们来解决第一种情况。由于 LFS 大约每 30 秒才写一次 CR，文件系统最后一个一致的快照（snapshot）可能已经相当旧了。因此，重启时，LFS 只需读入检查点区域（checkpoint region）、它所指向的 imap 片段以及后续的文件和目录，就能轻松恢复；但是，最后许多秒的更新会丢失。

为了改进这一点，LFS 会尝试用数据库领域所说的**前滚**（roll forward）技术，重建其中许多段。基本思想是：从最后一个检查点区域（checkpoint region）开始，找到日志（log）的末尾（它包含在 CR 中），然后据此读取后面的段，看看其中是否有有效的更新。如果有，LFS 就相应地更新文件系统，从而恢复自上一个检查点（checkpoint）以来写入的大部分数据和元数据。详见 Rosenblum 获奖的博士论文 [R92]。

43.13 小结

LFS 提出了一种更新磁盘的新方法。LFS 不原地覆盖文件，而总是写到磁盘上未使用的部分，以后再通过清理回收旧空间。这种方法在数据库系统中叫作**影子分页**（shadow paging）[L77]，在文件系统领域有时叫作**写时复制**（copy-on-write），它能让写入非常高效，因为 LFS 可以把所有更新收集到内存中的一个段里，然后一起顺序地写出去。

LFS 产生的大块写入，在许多不同的设备上都有出色的性能。在硬盘上，大块写入能把定位时间降到最低；在 RAID-4 和 RAID-5 这类基于奇偶校验（parity）的 RAID 上，它们完全避开了小写入问题（small-write problem）。最近的研究甚至表明，要在基于闪存（flash）的 SSD 上获得高性能，也需要大块的 I/O [HK+17]；因此，或许出人意料的是，LFS 风格的文件系统即使对这些新介质来说，也可能是一个极好的选择。

这种方法的缺点是会产生垃圾；数据的旧副本散落在磁盘各处，如果想回收这些空间供以后使用，就必须定期清理旧段（old segment）。清理成了 LFS 中许多争议的焦点，对清理开销的担忧 [SS+95] 或许限制了 LFS 最初在这一领域的影响。然而，一些现代的商用文件系统，包括 NetApp 的 **WAFL** [HLM94]、Sun 的 **ZFS** [B07] 和 Linux **btrfs** [R+13]，甚至现代的**基于闪存的 SSD** [AD14]，都采用了类似的写时复制（copy-on-write）方法来写入磁盘，因此 LFS 的思想遗产在这些现代文件系统中得以延续。特别是，WAFL 把清理问题变成了一个特性，从而绕开了它：它用**快照**（snapshot）提供文件系统的旧版本，用户不小心删除了当前文件时，随时都可以访问旧文件。

提示：化缺陷为优点

每当你的系统有一个根本性的缺陷时，看看能不能把它反过来变成一个特性或者有用的东西。NetApp 的 WAFL 就是这样处理旧文件内容的；WAFL 提供文件的旧版本，因此不必那么频繁地操心清理（尽管它最终还是会在后台删掉旧版本）；这一绝妙的转折，既带来了一个很酷的特性，又消除了 LFS 清理问题的大半。系统中还有其他这样的例子吗？毫无疑问，但你得自己去想了，因为这一章已经结束了，而且是大写的“结束”。结束。完事。玩完了。我们撤了。再见！

参考文献

[AD14] "Operating Systems: Three Easy Pieces" (Chapter: Flash-based Solid State Drives) by Remzi Arpaci-Dusseau and Andrea Arpaci-Dusseau. Arpaci-Dusseau Books, 2014. *A bit gauche to refer you to another chapter in this very book, but who are we to judge?*

[B07] "ZFS: The Last Word in File Systems" by Jeff Bonwick and Bill Moore. Copy Available: http://www.ostep.org/Citations/zfs_last.pdf. *Slides on ZFS; unfortunately, there is no great ZFS paper (yet). Maybe you will write one, so we can cite it here?*

[HK+17] "The Unwritten Contract of of Solid State Drives" by Jun He, Sudarsun Kannan, Andrea C. Arpaci-Dusseau, Remzi H. Arpaci-Dusseau. EuroSys '17, April 2017. *Which unwritten rules one must follow to extract high performance from an SSD? Interestingly, both request scale (large or parallel requests) and locality still matter, even on SSDs. The more things change …*

[HLM94] "File System Design for an NFS File Server Appliance" by Dave Hitz, James Lau, Michael Malcolm. USENIX Spring '94. *WAFL takes many ideas from LFS and RAID and puts it into a high-speed NFS appliance for the multi-billion dollar storage company NetApp.*

[L77] "Physical Integrity in a Large Segmented Database" by R. Lorie. ACM Transactions on Databases, Volume 2:1, 1977. *The original idea of shadow paging is presented here.*

[MJLF84] "A Fast File System for UNIX" by Marshall K. McKusick, William N. Joy, Sam J. Leffler, Robert S. Fabry. ACM TOCS, Volume 2:3, August 1984. *The original FFS paper; see the chapter on FFS for more details.*

[MR+97] "Improving the Performance of Log-structured File Systems with Adaptive Methods" by Jeanna Neefe Matthews, Drew Roselli, Adam M. Costello, Randolph Y. Wang, Thomas E. Anderson. SOSP 1997, pages 238-251, October, Saint Malo, France. *A more recent paper detailing better policies for cleaning in LFS.*

[M94] "A Better Update Policy" by Jeffrey C. Mogul. USENIX ATC '94, June 1994. *In this paper, Mogul finds that read workloads can be harmed by buffering writes for too long and then sending them to the disk in a big burst. Thus, he recommends sending writes more frequently and in smaller batches.*

[P98] "Hardware Technology Trends and Database Opportunities" by David A. Patterson. ACM SIGMOD '98 Keynote, 1998. Available online here: http://www.cs.berkeley.edu/~pattrsn/talks/keynote.html. *A great set of slides on technology trends in computer systems. Hopefully, Patterson will create another of these sometime soon.*

[R+13] "BTRFS: The Linux B-Tree Filesystem" by Ohad Rodeh, Josef Bacik, Chris Mason. ACM Transactions on Storage, Volume 9 Issue 3, August 2013. *Finally, a good paper on BTRFS, a modern take on copy-on-write file systems.*

[RO91] "Design and Implementation of the Log-structured File System" by Mendel Rosenblum and John Ousterhout. SOSP '91, Pacific Grove, CA, October 1991. *The original SOSP paper about LFS, which has been cited by hundreds of other papers and inspired many real systems.*

[R92] "Design and Implementation of the Log-structured File System" by Mendel Rosenblum. http://www.eecs.berkeley.edu/Pubs/TechRpts/1992/CSD-92-696.pdf. *The award-winning dissertation about LFS, with many of the details missing from the paper.*

[SS+95] "File system logging versus clustering: a performance comparison" by Margo Seltzer, Keith A. Smith, Hari Balakrishnan, Jacqueline Chang, Sara McMains, Venkata Padmanabhan. USENIX 1995 Technical Conference, New Orleans, Louisiana, 1995. *A paper that showed the LFS performance sometimes has problems, particularly for workloads with many calls to* `fsync()` *(such as database workloads). The paper was controversial at the time.*

[SO90] "Write-Only Disk Caches" by Jon A. Solworth, Cyril U. Orji. SIGMOD '90, Atlantic City, New Jersey, May 1990. *An early study of write buffering and its benefits. However, buffering for too long can be harmful: see Mogul [M94] for details.*

[Z+12] "De-indirection for Flash-based SSDs with Nameless Writes" by Yiying Zhang, Leo Prasath Arulraj, Andrea C. Arpaci-Dusseau, Remzi H. Arpaci-Dusseau. FAST '13, San Jose, California, February 2013. *Our paper on a new way to build flash-based storage devices, to avoid redundant mappings in the file system and FTL. The idea is for the device to pick the physical location of a write, and return the address to the file system, which stores the mapping.*

 © 2008–23, ARPACI-DUSSEAU

作业（模拟）

本节介绍 `lfs.py`，这是一个简单的 LFS 模拟器，可以帮你更好地理解基于 LFS 的文件系统是如何工作的。关于如何运行这个模拟器，请阅读 README。

问题

1. 运行 `./lfs.py -n 3`，也可以改变随机种子（`-s`）。你能推断出运行了哪些命令，才产生了最终的文件系统内容吗？你能说出这些命令是按什么顺序发出的吗？最后，你能判断出最终文件系统状态中每个块是否存活吗？用 `-o` 显示运行了哪些命令，用 `-c` 显示最终文件系统状态的存活情况。随着发出的命令数增加（即把 `-n 3` 改成 `-n 5`），这个任务对你来说会难多少？
2. 如果你觉得上面的题目很痛苦，可以显示每条命令引起的更新，给自己帮点忙。为此，运行 `./lfs.py -n 3 -i`。现在看看是否更容易理解每条命令是什么了。改变随机种子，得到不同的命令来解读（例如 `-s 1`、`-s 2`、`-s 3` 等）。
3. 为了进一步测试你判断每条命令对磁盘做了哪些更新的能力，运行 `./lfs.py -o -F -s 100`（也可以再试几个其他的随机种子）。这只会显示一组命令，**不会**显示文件系统的最终状态。你能推理出文件系统的最终状态一定是什么样子吗？
4. 现在，看看你能否判断出执行了一系列文件和目录操作之后，哪些文件和目录是存活的。运行 `./lfs.py -n 20 -s 1`，然后查看文件系统的最终状态。你能推断出哪些路径名（pathname）是有效的吗？运行 `./lfs.py -n 20 -s 1 -c -v` 查看结果。用 `-o` 运行，看看在给定的这一系列随机命令下，你的答案是否吻合。换用不同的随机种子，可以得到更多的题目。
5. 现在我们来发出一些具体的命令。首先，创建一个文件，并反复写入它。为此，使用 `-L` 选项，它可以让你指定要执行的具体命令。我们来创建文件 `/foo` 并写入它四次：`-L c,/foo:w,/foo,0,1:w,/foo,1,1:w,/foo,2,1:w,/foo,3,1 -o`。看看你能否判断出最终文件系统状态的存活情况；用 `-c` 检查你的答案。
6. 现在，我们来做同样的事，但用一次写操作代替四次。运行 `./lfs.py -o -L c,/foo:w,/foo,0,4` 来创建文件 `/foo`，并用一次写操作写入 4 个块。再计算一次存活情况，用 `-c` 检查你是否答对了。一次性写完一个文件（像这里这样）和一次写一个块的主要区别是什么？这说明了像真实的 LFS 那样在内存中缓冲更新有多重要？
7. 我们再来做一个具体的例子。首先，运行 `./lfs.py -L c,/foo:w,/foo,0,1`。这组命令做了什么？现在，运行 `./lfs.py -L c,/foo:w,/foo,7,1`。这组命令又做了什么？两者有什么不同？从这两组命令中，你能看出 inode 中的 `size` 字段有什么特点？
8. 现在我们来专门看看创建文件和创建目录的区别。依次运行模拟程序 `./lfs.py -L c,/foo` 和 `./lfs.py -L d,/foo`，分别创建一个文件和一个目录。这两次运行有什么相同之处，又有什么不同之处？

9. LFS 模拟器也支持硬链接（hard link）。运行下面的命令，研究它们是怎么工作的：`./lfs.py -L c,/foo:l,/foo,/bar:l,/foo,/goo -o -i`。
创建硬链接时会写出哪些块? 这和创建一个新文件有什么相似之处，又有什么不同? 创建链接时，引用计数（reference count）字段是如何变化的?

10. LFS 会做出许多不同的策略决定。我们在这里不会探讨其中的很多——或许留待以后——但这里有一个我们要探讨的简单策略：inode 号的选择。首先，运行 `./lfs.py -p c100 -n 10 -o -a s`，看看采用“顺序”分配策略（allocation policy）时的通常行为，这种策略会尽量使用最接近零的空闲 inode 号。然后，运行 `./lfs.py -p c100 -n 10 -o -a r`，改用“随机”策略（`-p c100` 选项确保随机操作百分之百都是文件创建）。随机策略和顺序策略会在磁盘上造成什么样的差异? 这说明在真实的 LFS 中，选择 inode 号有多重要?

11. 我们一直假设的最后一件事是，LFS 模拟器每次更新之后都会更新检查点区域（checkpoint region）。在真实的 LFS 中情况并非如此：为了避免长距离寻道，检查点区域（checkpoint region）是定期更新的。运行 `./lfs.py -N -i -o -s 1000`，看看在不强制把检查点区域（checkpoint region）写到磁盘时，一些操作以及文件系统的中间状态和最终状态。如果检查点区域（checkpoint region）从不更新，会发生什么? 如果它定期更新呢? 你能想出如何通过在日志中前滚（roll forward），把文件系统恢复到最新状态吗?